

ISEP — Instituto Superior de Engenharia do Porto
Pós-Graduação em Big Data & Data Science

Forest Risk Monitoring System

Plataforma integrada de deteção e previsão de risco de incêndio florestal

ODS 15 — Vida Terrestre
ODS 9 — Inovação e
Infraestruturas

Documento de projeto — Pós-Graduação ISEP 2024/2025

1. Contexto e problema.....	3
1.1 Os três problemas centrais.....	3
2. Solução proposta.....	3
2.1 Alinhamento com os ODS.....	3
3. Arquitetura técnica — as 6 camadas.....	4
3.1 Camada 1 — Ingestão de dados.....	4
Tecnologia: Apache Kafka.....	4
Topics criados.....	4
Fontes de dados públicas.....	4
3.2 Camada 2 — Processamento em tempo real.....	4
Tecnologia: Apache Spark Structured Streaming.....	4
Lógica implementada.....	5
3.3 Camada 3 — Armazenamento persistente.....	5
Tecnologia: Apache Cassandra + AWS S3.....	5
Schema Cassandra.....	5
Data lake S3.....	5
3.4 Camada 4 — Machine learning preditivo.....	5
Tecnologia: Python (scikit-learn / XGBoost) + Spark MLlib.....	5
Pipeline CRISP-DM.....	5
3.5 Camada 5 — Visualização & dashboards.....	6
Tecnologia: Power BI.....	6
3.6 Camada 6 — Segurança & infraestrutura.....	6
Tecnologia: AWS IAM + VPC + encriptação.....	6
4. Roadmap de desenvolvimento.....	7
Papéis do grupo.....	7
5. Alinhamento com as unidades curriculares.....	9
6. Entregáveis do projeto.....	9
6.1 Sistema funcional.....	9
6.2 Relatório técnico (UC Seminars).....	9
6.3 Apresentação oral (UC Seminars).....	10
7. Infraestrutura local — Docker Compose.....	10
Comando de arranque.....	10

1. Contexto e problema

Portugal é um dos países europeus mais afetados por incêndios florestais. A dimensão do problema não é, contudo, falta de dados — é a ausência de integração entre eles. Satélites da NASA detetam hotspots térmicos em tempo real, o IPMA mede temperatura, humidade e vento a cada hora, e o ICNF mantém cartografia florestal detalhada de todo o território. Estas fontes existem, são públicas e são gratuitas. O problema é que vivem em silos separados, são consultadas manualmente, e quando um decisor as cruza já passou tempo suficiente para um incêndio se propagar.

1.1 Os três problemas centrais

Problema 1 — Fragmentação dos dados

As fontes existem mas não comunicam entre si. Não há nenhum sistema que ingira simultaneamente dados meteorológicos, de satélite e de terreno e os trate como um fluxo contínuo e unificado.

Problema 2 — Reatividade em vez de previsão

Os sistemas atuais de alerta são maioritariamente reativos — detetam o incêndio quando já existe. O verdadeiro valor está em identificar as condições que tornam um incêndio provável nas próximas 24 a 72 horas, zona a zona, antes que aconteça.

Problema 3 — Informação não acionável

Mesmo quando os dados existem, chegam a técnicos em ficheiros CSV ou portais estáticos. Não há um dashboard operacional que um coordenador de proteção civil consiga abrir e perceber em 10 segundos onde está o risco mais alto hoje.

2. Solução proposta

O Forest Risk Monitoring System é uma pipeline de inteligência florestal — da leitura de sensor à decisão humana. O sistema unifica dados de múltiplas fontes (Kafka), processa-os em tempo real (Spark Streaming), prevê o risco futuro com machine learning (Python + MLlib), e entrega a informação de forma acionável (Power BI). Resolve os três problemas identificados em sequência.

2.1 Alinhamento com os ODS

Camada	Tecnologia	Responsabilidade
ODS 15	Vida Terrestre	O sistema protege ecossistemas florestais através da deteção precoce de risco. Permite

		intervenção antes do incêndio, reduzindo área ardida e preservando biodiversidade.
ODS 9	Inovação e Infraestruturas	A infraestrutura Big Data é ela própria a inovação. Kafka, Spark, Cassandra e ML constituem uma infraestrutura tecnológica de ponta aplicada a um problema de impacto social real.

3. Arquitetura técnica — as 6 camadas

O sistema é organizado em seis camadas funcionais, cada uma com responsabilidade bem delimitada. As camadas estão desacopladas por design — cada uma pode evoluir independentemente sem quebrar as outras.

3.1 Camada 1 — Ingestão de dados

Tecnologia: Apache Kafka

O Kafka atua como o sistema nervoso central do projeto. Recebe eventos em tempo real de múltiplas fontes heterogêneas e os distribui de forma fiável para as camadas seguintes, garantindo que nenhum evento se perde mesmo que o consumidor esteja temporariamente indisponível.

Topics criados

- sensor-events — leituras de sensores IoT (temperatura, humidade, velocidade do vento), com 3 partições
- satellite-hotspots — deteções de hotspots térmicos da NASA FIRMS, com 3 partições
- weather-data — dados meteorológicos do IPMA, com 3 partições
- fire-alerts — alertas gerados pelo motor de risco, com 1 partição e retenção de 30 dias

Fontes de dados públicas

- NASA FIRMS (Fire Information for Resource Management System) — hotspots em tempo real
- IPMA API — temperatura, humidade, velocidade e direção do vento
- ICNF — cartografia de uso do solo e tipo de vegetação
- Sensores IoT simulados — script Python que gera leituras realistas com variação estatística

3.2 Camada 2 — Processamento em tempo real

Tecnologia: Apache Spark Structured Streaming

O Spark é o motor de processamento que transforma eventos brutos em inteligência. Consome os topics Kafka em micro-batches de 30 segundos e aplica lógica de detecção de anomalias em janelas temporais deslizantes.

Lógica implementada

- Sliding window de 10 minutos com slide de 5 minutos sobre cada célula geográfica
- Join entre os três streams (sensores + satélite + meteorologia) com watermark de 2 minutos
- Cálculo do índice de risco composto (0–100) com ponderação configurável
- Regra de alerta imediato: temperatura > 35°C E humidade < 20% E vento > 30 km/h
- Publicação de alertas no topic fire-alerts com nível (LOW / MEDIUM / HIGH / CRITICAL)

3.3 Camada 3 — Armazenamento persistente

Tecnologia: Apache Cassandra + AWS S3

Dois tipos de armazenamento com papéis complementares: o Cassandra para hot data de alta velocidade (leituras e alertas das últimas 72h), e o S3 para o data lake histórico de longo prazo (dados para treino dos modelos ML).

Schema Cassandra

- Tabela sensor_readings — particionada por (grid_id, hour_bucket), ordenada por event_time DESC. Retenção de 72h.
- Tabela fire_alerts — particionada por grid_id, ordenada por alert_time DESC. Sem TTL.
- Tabela risk_predictions — previsões ML por célula e horizonte temporal (24h, 48h, 72h).

Data lake S3

- Bucket forest-risk-datalake — dados históricos em formato Parquet, particionados por ano/mês/zona
- Bucket forest-risk-models — versões dos modelos ML serializados com MLflow

3.4 Camada 4 — Machine learning preditivo

Tecnologia: Python (scikit-learn / XGBoost) + Spark MLlib

A camada que transforma dados históricos em previsões de risco. Segue rigorosamente a metodologia CRISP-DM ensinada na UC de Data Mining em Python, e os modelos treinados são integrados no pipeline Spark para geração automática de previsões.

Pipeline CRISP-DM

- Business understanding — definir métricas de sucesso (F1 > 0.80 na classe HIGH)
- Data understanding — EDA sobre 5 anos de dados NASA FIRMS + IPMA

- Data preparation — feature engineering: grid agregado, médias móveis, índices compostos
- Modeling — treino de Random Forest e XGBoost com validação cruzada
- Evaluation — comparação de F1-score, ROC-AUC, matriz de confusão
- Deployment — modelo exportado para Spark MLlib, retreino semanal automático

3.5 Camada 5 — Visualização & dashboards

Tecnologia: Power BI

Três dashboards distintos com públicos-alvo diferentes: operacional (protecção civil), preditivo (planeamento), e analítico (investigação/relatórios).

- Dashboard tempo real — mapa de Portugal com células 5×5 km coloridas por risco atual, atualização a cada 5 minutos
- Dashboard preditivo — probabilidades de risco para as próximas 24–72h, top-10 zonas de maior risco
- Dashboard histórico — área ardida por ano/mês/distrito, correlações, métricas do modelo ML

3.6 Camada 6 — Segurança & infraestrutura

Tecnologia: AWS IAM + VPC + encriptação

Transversal a todo o sistema. Garante que os dados estão protegidos, que o acesso é controlado por papel, e que o sistema cumpre o RGPD — relevante dado que coordenadas de espécies protegidas têm protecção legal.

- AWS IAM com roles separadas para ingestão, processamento e visualização (least-privilege)
- VPC privada — Kafka e Cassandra não expostos à internet diretamente
- TLS 1.3 em todos os canais Kafka; SSE-S3 no data lake; encriptação de disco no Cassandra
- Data retention policy: dados pessoais 90 dias; dados ambientais retidos indefinidamente
- Audit log imutável via topic Kafka dedicado e AWS CloudTrail

4. Roadmap de desenvolvimento

O projeto é desenvolvido em 5 fases ao longo de 14 semanas por um grupo de 3 elementos com papéis complementares. A fase 3 (semana 8) é o marco crítico — se o sistema core estiver funcional nessa altura, a entrega final está garantida.

Papéis do grupo

- Pessoa A — Data Engineer: responsável pela pipeline de dados (Kafka, Cassandra, Spark jobs)
- Pessoa B — Data Scientist: responsável pelos modelos ML e qualidade dos dados (CRISP-DM)
- Pessoa C — BI & DevOps: responsável pela infraestrutura cloud e visualização (Power BI, AWS)

Fase 1 — Setup & dados Semanas 1–2		
A — Data Engineer	B — Data Scientist	C — BI & DevOps
Criar repositório Git com estrutura de pastas	Download histórico NASA FIRMS (5 anos Portugal)	Criar conta AWS Free Tier + bucket S3
Instalar Docker + Docker Compose (Kafka, Cassandra)	Exploração inicial dos dados (pandas, EDA)	Configurar IAM roles e políticas de acesso
Escrever producers Python para NASA FIRMS e IPMA	Definir schema de dados e variáveis relevantes	Documentar decisões de arquitetura no README

Fase 2 — Pipeline de ingestão completa Semanas 3–5		
A — Data Engineer	B — Data Scientist	C — BI & DevOps
Configurar Kafka com 4 topics e partições	Producer de sensores IoT simulados (Python)	Job Spark batch Cassandra → Parquet no S3
Schema Cassandra — tabelas particionadas por grid_id	Validar qualidade dos dados — missing values, outliers	Monitorização básica: logs Kafka, health Cassandra
Consumer Kafka → escreve em Cassandra em tempo real	Feature engineering: criar grid 5×5 km para Portugal	Primeiro Power BI conectado ao S3 — tabela bruta

Fase 3 — Processamento em tempo real & alertas Semanas 6–8		
A — Data Engineer	B — Data Scientist	C — BI & DevOps
Spark Structured Streaming — ler topics Kafka	Fórmula do índice de risco composto (0–100)	Topic fire-alerts → persistência em Cassandra e S3
Sliding window de 10 min com agregações por grid_id	Regras de alerta: temp > 35°C + hum. < 20% + vento > 30	Mapa Power BI: células coloridas por risco atual
Join entre 3 streams com watermark de 2 minutos	Testes com dados históricos de 2017 e 2022	Atualização automática a cada 5 minutos

Marco intermédio — Semana 8

Se nesta semana o sistema core está funcional (dados a entrar no Kafka, alertas a sair, mapa a atualizar), a entrega final está garantida. As fases 4 e 5 são valor acrescentado e polimento.

Fase 4 — Machine learning preditivo Semanas 9–11

A — Data Engineer	B — Data Scientist	C — BI & DevOps
Job Spark batch para preparar dataset de treino do S3	Pipeline CRISP-DM completa: EDA → features → modelo	Dashboard preditivo Power BI: mapa 24–72h
Integrar previsões do modelo no pipeline streaming	Treinar Random Forest e XGBoost, comparar métricas	Gráfico de evolução do risco por distrito
Persistir previsões 24h em nova tabela Cassandra	Análise de importância de features	Tabela top-10 zonas com maior risco previsto

Fase 5 — Relatório, testes e apresentação Semanas 12–14

A — Data Engineer	B — Data Scientist	C — BI & DevOps
Testes de carga: 10.000 eventos/segundo no Kafka	Validação do modelo com dados de 2023 (não vistos)	Polir dashboards Power BI — modo apresentação
Documentar arquitetura técnica com diagramas	Escrever secções 4–5: CRISP-DM e métricas	Escrever secções 6–7: segurança e conclusões
Escrever secções 1–3 do relatório	Preparar slides da componente ML	Ensaio geral — demo ao vivo do sistema

5. Alinhamento com as unidades curriculares

Cada UC do curso contribui de forma orgânica para o projeto — não como exercícios forçados, mas como componentes que seriam necessárias independentemente. A tabela seguinte mapeia cada camada técnica à UC que a suporta.

Camada	Tecnologia	Responsabilidade
Big Data Tools I	Cassandra	Design do schema de séries temporais, estratégia de particionamento, queries CQL, gestão de TTL e retenção de dados.
Big Data Tools II	Kafka + Spark	Configuração do broker Kafka, criação de topics, producers/consumers Python, Spark Structured Streaming, sliding windows.
Data Mining em Python	scikit-learn / XGBoost	Pipeline CRISP-DM completa, feature engineering, treino e avaliação de modelos de classificação, análise de importância de features.
Business Intelligence	Power BI	Três dashboards: tempo real com mapa de risco, preditivo com horizonte 72h, histórico com KPIs e análise de tendências.
Data Security & Privacy	AWS IAM / TLS / RGPD	Infraestrutura AWS (VPC, IAM, S3), encriptação at-rest e in-transit, audit logging, conformidade RGPD ambiental.
Seminars	Projeto integrador	Relatório técnico (7 secções), apresentação oral com demo ao vivo, repositório Git documentado com instruções de deploy.

6. Entregáveis do projeto

6.1 Sistema funcional

- Pipeline completa a correr em Docker — arranca com um único comando (docker compose up)
- Repositório Git com código documentado, README e instruções de deploy
- Demo ao vivo: injeção de evento de risco alto → alerta no dashboard em < 60 segundos

6.2 Relatório técnico (UC Seminars)

- Secção 1 — Introdução: contexto, problema, objetivos e ODS
- Secção 2 — Estado da arte: sistemas existentes de monitorização florestal
- Secção 3 — Arquitetura: as 6 camadas técnicas com diagramas de blocos
- Secção 4 — Implementação: decisões técnicas e trade-offs
- Secção 5 — Modelos ML: CRISP-DM, métricas, resultados
- Secção 6 — Segurança: infraestrutura AWS, encriptação, RGPD

- Secção 7 — Conclusões: limitações, trabalho futuro, impacto no ODS 15

6.3 Apresentação oral (UC Seminars)

- Duração estimada: 20 minutos de apresentação + 10 minutos de Q&A
- Estrutura: problema (3 min) → solução técnica (8 min) → demo ao vivo (6 min) → resultados ML (3 min)
- Demo ao vivo com o dashboard Power BI e injeção de eventos em tempo real

7. Infraestrutura local — Docker Compose

Todo o ambiente de desenvolvimento corre localmente via Docker Compose. O ficheiro `docker-compose.yml` incluído neste pacote sobe os seguintes serviços com um único comando:

Camada	Tecnologia	Responsabilidade
Kafka	porta 29092	Broker de mensagens. Acesso externo do host via localhost:29092. Kafka UI disponível em http://localhost:8080.
Zookeeper	porta 2181	Gestor de coordenação do Kafka. Normalmente não precisas de interagir diretamente.
Cassandra	porta 9042	Base de dados. Acesso via cqlsh: <code>docker exec -it cassandra cqlsh</code> . Schema criado automaticamente ao arrancar.
Spark Master	porta 8090	Spark UI disponível em http://localhost:8090. Submete jobs com <code>spark-submit</code> apontando para <code>spark://localhost:7077</code> .
Spark Worker	—	Worker com 1GB RAM e 2 cores. Escala adicionando mais workers no <code>docker-compose.yml</code> .
LocalStack S3	porta 4566	Emulação local do AWS S3. Usa <code>awslocal s3 ls</code> para listar buckets. Buckets criados automaticamente ao arrancar.
Kafka UI	porta 8080	Interface web para visualizar topics, mensagens e consumer groups. Muito útil para debug.

Comando de arranque

Arrancar o ambiente completo

`docker compose up -d` Verificar estado: `docker compose ps` Parar (mantendo dados): `docker compose down` Parar e limpar tudo: `docker compose down -v`

— *Fim do documento* —